

Capstone: AICD

Final Report, September 2025



Authors:

Zubair Elliot

ELLZUB001

Meekaaeel Booley

BLYMEE001

Mubashir Dawood

DWDMUB001

Contents

Contents

Abstract	3
1. Introduction	3
1.1 Problem Statement	3
1.2 Project Scope	3
1.3 Approach	4
2.1 Requirements Captured	5
2.1.1 Functional Requirements	5
2.1.2 Non-Functional Requirements	5
2.1.3 Usability Requirements	6
2.1.4 Use Case Narratives	6
2.1.5 Use Case Diagrams	7
2.2 Design Overview	8
System Architecture	8
Key Design Decisions	8
2.3 Implementation	9
Backend Implementation	9
Frontend Implementation	9
2.4 Program Validation and Verification	9
Quality Management Plan	9
Testing Implementation	10
Performance Validation	10
Accuracy Assessment	10
Conclusion	10
Appendix A - User Manual	11
Appendix B - API Endpoints Reference	13
Authentication	13
Core Endpoints	14
Appendix C - Test Documentation	18
Model Testing Suite (test_model.py)	18

API Testing Suite (test_api_func.py)	18
Frontend Component Testing (Storybook)	19
Integration Testing.....	19
Overall Test Summary	19
References.....	20

Abstract

This project addresses the ever-lasting battle of detecting artificial generated content, an issue that has become a consistent enemy to authentic work and transparency. The aim was to develop a full stack web application that classifies whether user-submitted text was human or AI-generated, providing a probability score, confidence score as well as highlighting suspected AI-generated text. The solution was created by using a modern web architecture comprised of React+Vite frontend, Flask API layer, Redis Backend and fine-tuning an ELECTRA based model for text classification. The system supports core user interactions through an intuitive interface, while storing results in a database for research purposes.

1. Introduction

Due to the increasing popularity of large language models such as ChatGPT, there has been a range of both opportunities and challenges created in the academic world. While these models have their benefits, it also creates a moral dilemma around academic integrity, author verification and damages the trustworthiness of digital information. In response to these concerns, detection systems have become an important tool for identifying whether text is authentic or artificial.

1.1 Problem Statement

The rise of large language models such as ChatGPT has seen an increase in AI generated content, while many see the positive has this enhancing the production many things such as content writing, it also creates a fine line of concern around the integrity of the generated content. The core challenge lies in differentiating between what is considered human like text and artificially generated text.

1.2 Project Scope

The purpose of this project is to develop a full-stack web application that detects AI-generated content in user-submitted text. Our application aims to provide a fast and lightweight solution in detecting AI-generated content with a locally hosted open-source language model, the system will analyse user-submitted text and classify as either AI-generated or human-written. Open-source models include roberta-base-openai-detector', 'GPT-2', and 'DistilBERT', all of which are accessible via Hugging Face and compatible with PyTorch or

TensorFlow frameworks. These models are widely used in the NLP community and can be fine-tuned or extended with additional data. The model will output a confidence score alongside a textual verdict. Additionally, part of the purpose of this project is to design the detection system in a modular, extensible way so that future students or researchers (e.g., master's students) can build upon the current system. The project is intended not only to fulfil current functionality requirements but also to serve as a foundation for further academic development and research. Key to the project is the focus on a transparent use of artificial intelligence. By avoiding reliance on ready-made APIs and instead implementing our own detection logic in Python using libraries like PyTorch and Hugging Face Transformers, we contribute to open and explainable AI practices. Additionally, all user inputs and results will be stored in a database to enable potential research, auditability, and model improvement over time. The system will be structured around a modern web architecture: a React based frontend for user interaction, a Redis backend for business logic and database interaction, and a Flask-based intermediary API for handling communication with the AI model.

1.3 Approach

We began the approach by adopting an evolutionary prototyping methodology. Development began with a vertical prototype focusing on basic detection, which was supported by a minimalist interface, this ensured that we focused on the core principle of the project, which was AI detection.

A fundamental part of this approach was the amount of training the detector received. Datasets were gathered from Hugging Face, Kaggle, GitHub repositories, and academic textbooks. Once collected, the data underwent preprocessing and was used to train transformer-based models. The models were repeatedly fine-tuned and evaluated on Google Colab, with adjustments made to hyperparameters and thresholds until the system produced satisfactory levels of accuracy, recall, and F1-score. This iterative process ensured that the final detector was both reliable and robust when distinguishing between AI-generated and human-written text.

In parallel, the frontend was built with React and Vite, using Storybook for component-based UI development and testing, while the backend was implemented with Redis and Flask to maintain clear separation of concerns. The AI model was integrated through Hugging Face's libraries, allowing for fine-tuning and extensibility.

The team employed a blend of traditional software engineering practices and agile principles. Formal analysis and modelling were completed in the planning stages, including use-case design, class modelling, and test strategy development. Agile practices were then applied during implementation, with cyclical feedback loops, peer code reviews, and continuous integration via GitLab Actions. This hybrid approach ensured both academic integrity and practical adaptability, resulting in a system that is functional, transparent, and extensible.

2.1 Requirements Captured

This section outlines the requirements of the AI Detector web app. The system is designed to analyse and classify input text to determine if the input is AI-generated or human-written. The requirements are categorised into functional requirements, describing specific behaviours, and non-functional requirements, which outline the attributes and performance constraints of the system.

2.1.1 Functional Requirements

These describe what the system should do, specific features and behaviours. They directly address user needs and system functions.

- *Session Management*
 - The system will create and maintain a unique session for each user, using cookies.
 - The system will allow users to retrieve their detection history based on their session cookie.
 - The system will allow users to clear their detection history from Redis.
- *Input handling*
 - The system will provide a text box for users to manually input text for AI detection.
 - The system will support file uploads (.txt, .pdf, .docx) for text analysis and AI detection.
 - The system will extract and process text from uploaded files.
- *AI Detection*
 - The system will classify input as either AI-generated or human-written.
 - The system will produce a confidence score for each detection result
 - The system will display detection results as after processing input.
- *User input and Feedback*
 - The system will return AI generated text as highlighted text.
 - The system will notify users when unsupported files are uploaded.
 - The system will notify users if text has too little words.

2.1.2 Non-Functional Requirements

This describes how the system should work, discussing qualities, constraints and performance goals.

- *Performance*
 - The Redis database allows fast storage and lookups.
 - The system will process and return detection results within 2 seconds for text input under 1000 words.
- *Scalability*
 - The system will be built for easy scalability for those who want to pick up the project.
- *Security*
 - The system will enforce HTTP for all client-server communication.
 - The system will restrict file uploads to safe formats (.txt, .pdf, .docx) and scan text for malicious content.
 - The system will restrict file size to 500kb maximum.
 - The system will validate uploaded file names to remove special characters that may cause risk.

- *Maintainability*
 - o The system will be developed using modular architecture, allowing developers to understand it with ease.
 - o The system will be separated into parts, allowing updates to the AI model without affecting session management logic.
 - o Includes easy to read documents and comments for understanding.
- *Reliability and Availability*
 - o In case of Redis failure, in system memory will run for a browser session until Redis can reconnect.
 - o The system will display informative error messages instead of technical errors.
 - o The system will persist session data in Redis.

2.1.3 Usability Requirements

The usability requirements make sure that the AI Detector app is easy to use and intuitive.
(See Appendix A - User Manual)

- The system will provide a simple and intuitive interface with a clear text box for manual input.
- The system will allow button-based file uploads.
- The system will display detection results in clear visual metrics.
- The system will provide progress indicators for file uploads and text analysis.
- The system will have clear navigation from the history page to detection page and home page.

2.1.4 Use Case Narratives

Primary Actors: Anonymous Users (e.g. Student, Lecturer, etc.)

Secondary Actor: System Admin

Use Case 1: Analyse Text for AI Generated Content

- Precondition: User has an active session (cookie)
- Steps:
 1. User navigates to input page.
 2. User enters text into the text box or uploads file.
 3. User clicks the “Detect” button.
 4. System sanitises and validates the input text.
 5. System then analyses the text using the AI detection model.
 6. System stores result in Redis and linked to the user’s session.
 7. System returns detection results and confidence score.
- Postcondition: Detection results are displayed and history stored.

Use Case 2: View detection results and Feedback

- Precondition: User has session with stored history
- Steps:
 1. User navigates to history page.
 2. System displays list of history with timestamps.
 3. User clicks on specific history save.
 4. System retrieves history from Redis using the session cookie.
 5. System displays feedback with confidence score.

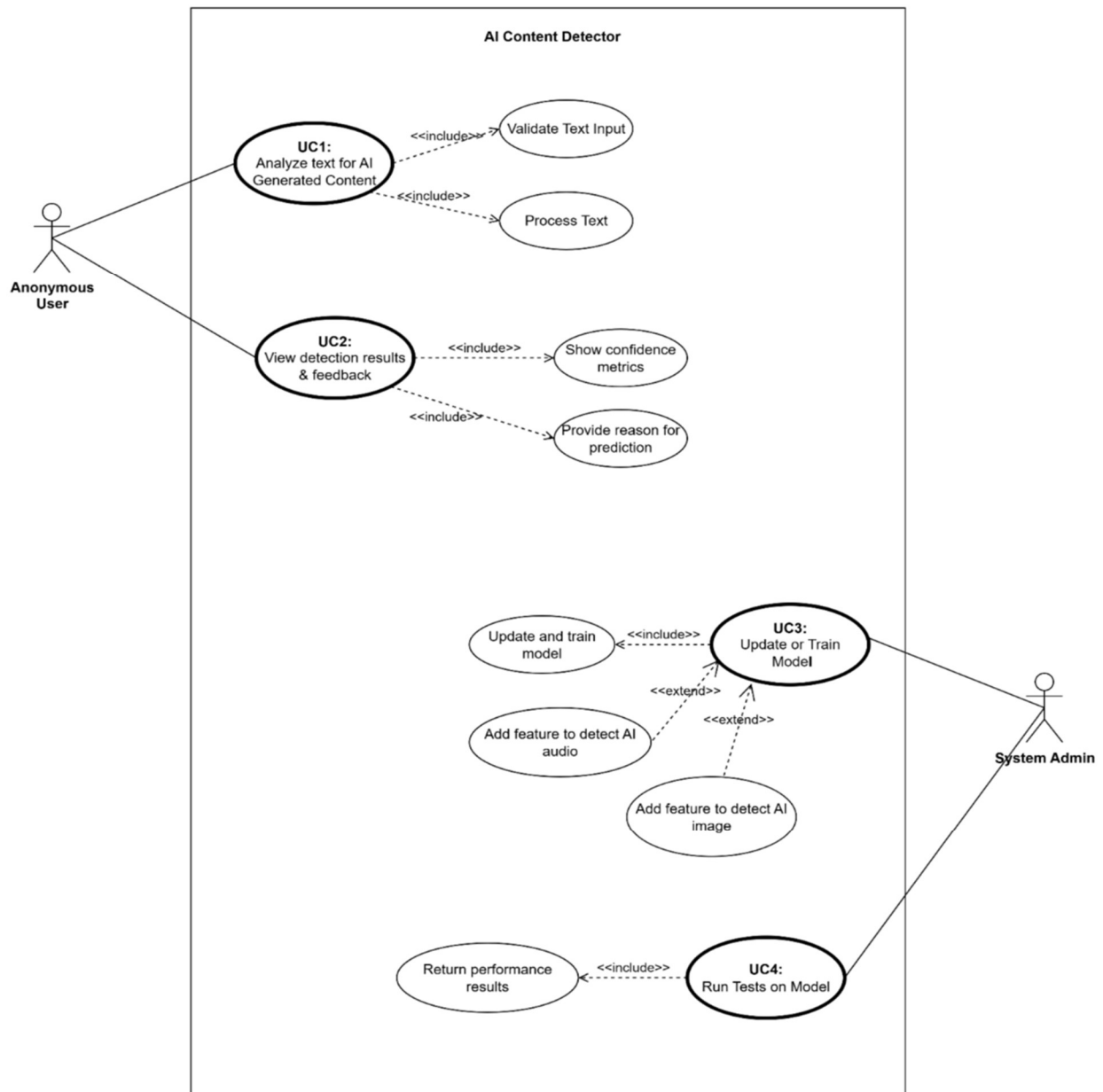
- Postcondition: User can view their detection history with feedback.

2.1.5 Use Case Diagrams

Primary Actors: Anonymous Users (e.g. Student, Lecturer, Employer, Employee)

Secondary Actor: System Admin (LLM maintenance)

Goal: Allow users to detect if text submitted has been generated by AI



2.2 Design Overview

System Architecture

The AI Content Detector follows a modular monolithic architecture that separates concerns and enables independent development and deployment of components.

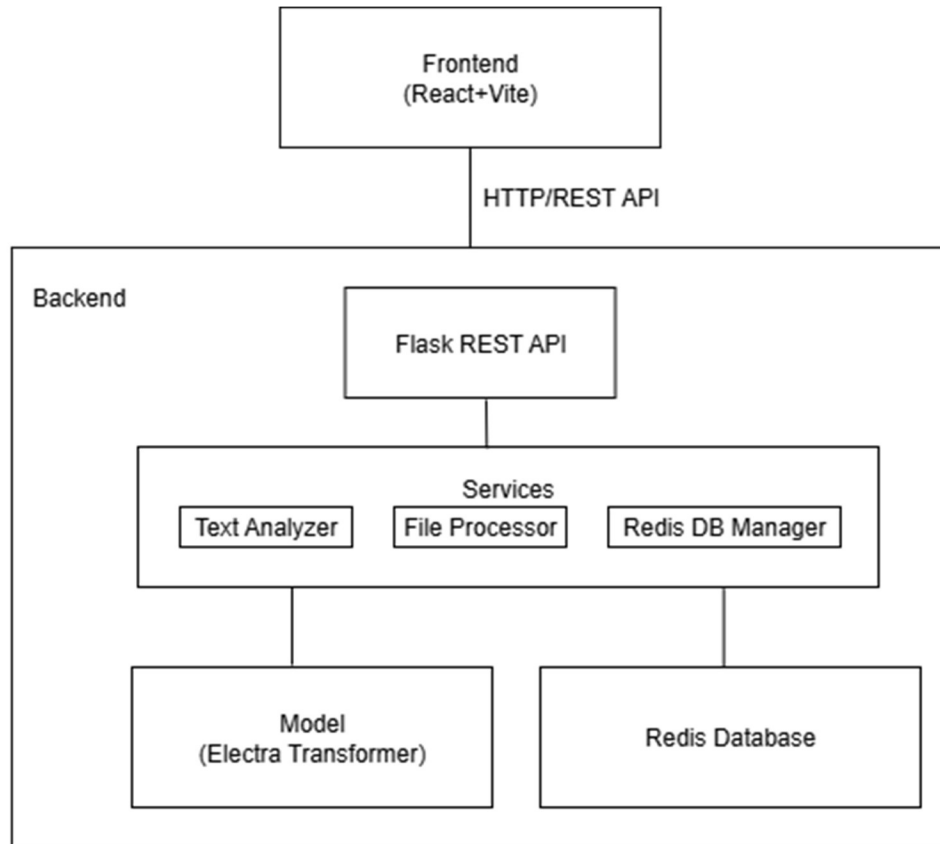


Figure 1: System Architecture Diagram showing the design with clear separation of concerns.

Key Design Decisions

Transformer Model Selection:

The Electra model was chosen for its efficiency compared to BERT while maintaining strong performance on text classification tasks.

Intelligent Analysis Strategy:

The system automatically selects between single-text and sentence-level analysis based on content structure. This hybrid approach optimizes both performance and accuracy.

Session Persistence:

Redis provides fast, reliable session storage with automatic fallback to in-memory storage, ensuring system availability even when Redis is unavailable.

Modular Design:

Each component handles specific responsibilities (file processing, text analysis, session management) enabling independent testing, development, and deployment.

2.3 Implementation

Backend Implementation

AI Detection Model (model.py):

This component wraps the Electra transformer model, providing simplified interfaces for prediction, batch processing, and confidence assessment. The model loads from disk with (CPU/GPU) and handles tokenization.

Text Analysis Engine (text_analyser.py):

The analysis engine coordinates the entire detection process, implementing intelligent splitting algorithms and result aggregation.

File Processing Service (file_processor.py):

This service handles secure file uploads with validation, temporary storage, and format-specific text extraction. It supports PDF (PyPDF2), DOCX (python-docx), and TXT files with encoding detection.

Session Management (redis_manager.py):

The Redis manager provides persistent storage with automatic serialization of complex json data structures. It includes connection management, error handling, and fallback strategies.

Frontend Implementation

React Component Architecture:

The frontend employs a component-based architecture with reusable UI elements. Key components include TextInput with file attachment support, Metric displays for confidence visualization, and HighlightedText for result presentation.

State Management:

The application uses React hooks for state management, maintaining analysis results, loading states, and error conditions. The design ensures consistent user experience across different interaction patterns.

API Integration:

The frontend communicates with the backend through a comprehensive API client (api_caller.js) that handles authentication, session management, and error processing.

2.4 Program Validation and Verification

Quality Management Plan

The testing strategy follows multiple levels of validation to ensure system reliability, accuracy, and robustness. Our approach combines automated testing with manual validation across different system components.

Table 1: Summary Testing Plan

Process	Technique
1. Model Testing: validate AI detection accuracy and consistency	Unit tests with known AI/human samples, edge case validation, batch processing verification.
2. API Testing: verify endpoint functionality and error handling.	Integration tests with authentication, session management, file upload validation
3. Component Testing: validate UI components and interactions	Storybook tests for React components, user interaction validation
4. System Testing: end-to-end functionality verification	Full workflow testing.

Testing Implementation

(See Appendix C)

Model Validation:

The core AI detection model underwent comprehensive testing including basic prediction validation, confidence scoring accuracy, batch processing consistency, and edge case handling (empty text, very long text, special characters). All tests achieved 100% pass rates with consistent results across multiple runs.

API Endpoint Testing:

All REST API endpoints were tested for authentication, request validation, response formatting, and error handling. The testing covered normal operation, boundary conditions, and failure scenarios. Session management, history tracking, and file upload functionality all performed as specified.

Frontend Component Testing:

React components were validated through Storybook testing, ensuring consistent rendering, proper user interactions, and responsive design. All 36 component tests passed validation.

Table 2: Summary of Test Results

Test Category	Normal Functioning	Edge Cases	Error Handling
Model Tests (9 tests) (See Appendix C)	Passed all prediction and confidence tests	Handled empty text, long text, special characters	Graceful error recovery
API Tests (16 tests) (See Appendix C)	All endpoints functional with authentication	File size limits, invalid formats handled	Proper HTTP status codes returned
Component Tests (36 tests) (See Appendix C)	All UI components render correctly	Responsive design validated	Error states displayed properly
Integration Tests (See Appendix C)	End-to-end workflows functional	Session persistence across requests	Fallback mechanisms operational

Performance Validation

Performance testing revealed that single text analysis completes in approximately 200-400ms on standard hardware. Memory usage remains stable during extended operation, and the system handles concurrent requests appropriately.

File processing performance varies by format: TXT files process instantly, DOCX files require 100-300ms for extraction, and PDF processing takes 200-500ms depending on document complexity. All processing times fall within acceptable user experience thresholds.

Accuracy Assessment

The AI detection model demonstrates high accuracy on test samples. The sentence-level analysis proves particularly effective for mixed AI/human content, providing granular insights that improve overall classification accuracy.

Conclusion

From the start of our project, our goal was to tackle the growing challenge of detecting AI-generated text in a transparent and academically valuable way. We wanted our final system to provide users with a reliable tool to identify whether text was authored by a human or an AI model, while also being extremely user friendly.

Our success revolves around the system we built that integrates a React-based frontend, a Flask API and backend, and transformer-based models sourced from Hugging Face. By training the detector on datasets collected from Hugging Face, Kaggle, GitHub, and academic textbooks, and refining the model through rigorous evaluation, we achieved strong classification performance with clear probability scoring, this allows users to not only detect AI generated content, but to also view feedback by the model.

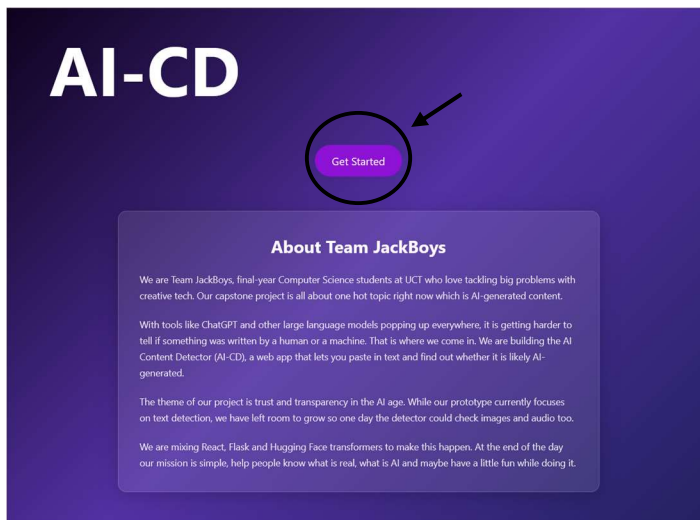
The accomplishments of this project include the development of a full-stack architecture, successful training, deployments of an AI text detector and the implementation of a user friendly interface. In addition, the system has been designed in a way that there is potential to enhance it with various forms of other detection as well as improving current text detection.

In conclusion, our project meets the core requirements with a modular design, robust performance, intuitive interface and a working AI detector. As a result, we view our final system as a significant success and hope it proves as valuable and engaging to its users as it was for us to develop.

Appendix A - User Manual

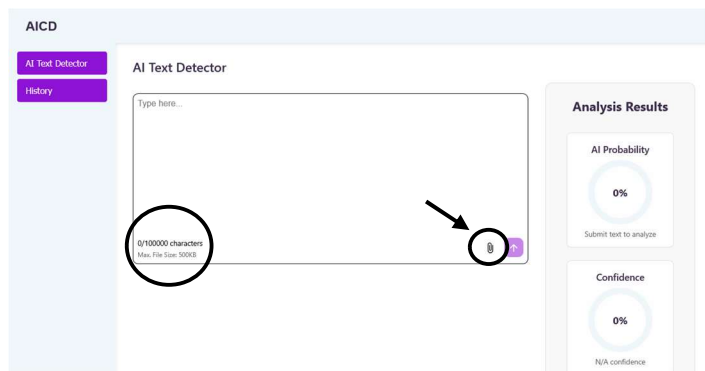
Step 1

- Navigate to detection page by clicking “Get started”.



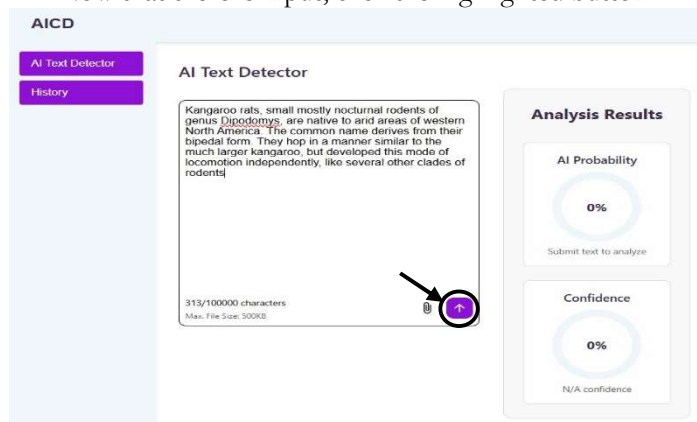
Step 2

- Input text via input box or file upload, adhering to limits.



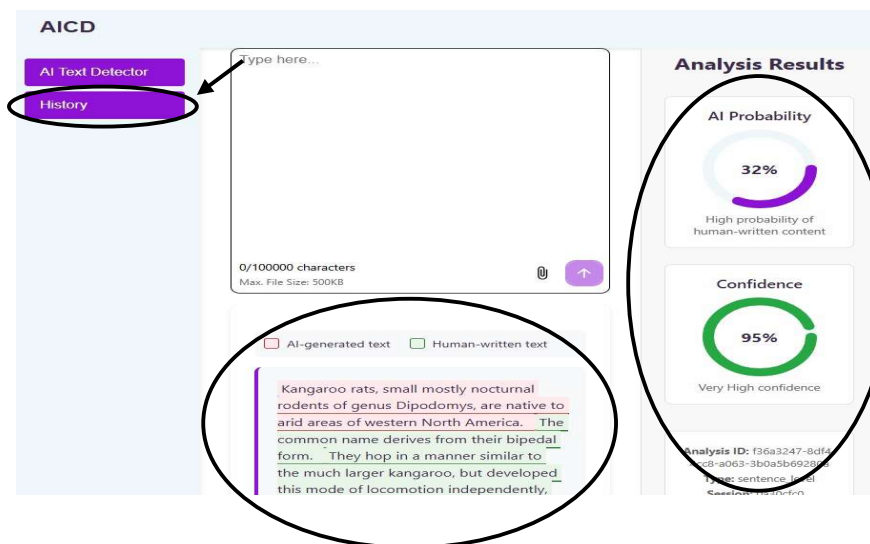
Step 3

- Now that there is input, click the highlighted button



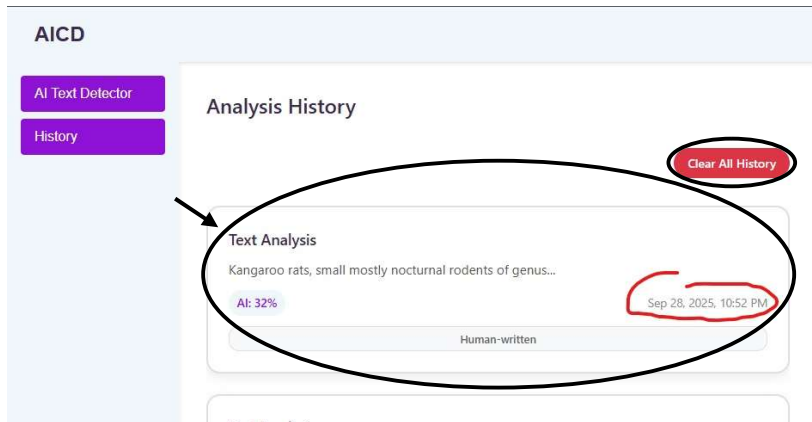
Step 3

- View feedback, to view history click history button.



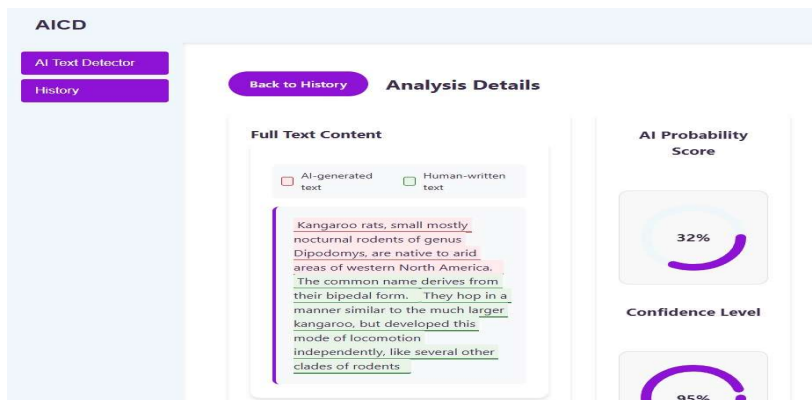
Step 4

- Click on specific history or clear all history.



Step 5

- View specific history.



Troubleshooting

Common Issues:

- "Text too short": Ensure input has at least 10 words
- "File too large": Reduce file size or use text extraction
- "Unsupported file": Use PDF, DOCX, or TXT formats only
- "Connection error": Check backend and redis is running

Appendix B - API Endpoints Reference

Authentication

All endpoints require authentication via API key provided in one of two ways:

- Header: X-API-Key: jackboys25
- Query parameter: ?api_key=jackboys25

Core Endpoints

Health Check

GET /api/health

Verifies API availability and returns system configuration.

Response (200 OK):

```
{
  "status": "healthy",
  "message": "AI Content Detector API is running",
  "version": "1.0.0",
  "endpoints": {
    "detect": "/api/detect",
    "history": "/api/history",
    "session": "/api/session"
  }
}
```

Text Analysis

POST /api/detect

Analyzes text for AI generation detection.

Request Headers:

Content-Type: application/json

X-API-Key: jackboys25

Request Body:

```
{
  "text": "Text to analyze here",
  "force_single_analysis": false
}
```

Parameters:

- text (string, required): Text content to analyze (10+ words, max 100,000 characters)
- force_single_analysis (boolean, optional): Force single-text analysis even for multi-sentence text

Response (200 OK) for Single Text:

```
{
  "success": true,
  "analysis_id": "550e8400-e29b-41d4-a716-446655440000",
  "analysis_type": "single_text",
  "result": {
    "ai_probability": 0.8234,
    "human_probability": 0.1766,
    "confidence": 0.8234,
    "classification": "AI-generated",
  }
}
```

```
"text_length": 150,
"source_type": "text"
}
}
```

Response (200 OK) for Sentence Level:

```
{
  "success": true,
  "analysis_id": "550e8400-e29b-41d4-a716-446655440001",
  "analysis_type": "sentence_level",
  "result": {
    "overall_ai_probability": 0.7456,
    "overall_human_probability": 0.2544,
    "overall_confidence": 0.8123,
    "overall_classification": "AI-generated",
    "sentence_count": 4,
    "analyzed_sentences": 4,
    "ai_sentence_count": 3,
    "human_sentence_count": 1,
    "ai_percentage": 75.0,
    "confidence_range": {
      "min": 0.6789,
      "max": 0.9234,
      "std_dev": 0.1023
    }
  },
  "sentence_results": [
    {
      "index": 0,
      "sentence_preview": "First sentence text...",
      "result": {
        "ai_probability": 0.9234,
        "human_probability": 0.0766,
        "confidence": 0.9234,
        "classification": "AI-generated"
      }
    }
  ]
}
```

Error Responses:

- 400 Bad Request: Invalid input (text too short, too long, or missing)

- 401 Unauthorized: Missing or invalid API key
- 413 Payload Too Large: File exceeds 500KB limit
- 500 Internal Server Error: Model or processing error

File Upload Analysis

POST /api/detect

Analyzes uploaded document files.

Request Headers:

Content-Type: multipart/form-data

X-API-Key: jackboys25

Request Body:

file: Binary file data

Supported Formats: PDF, DOCX, TXT (max 500KB)

Response: Same format as text analysis endpoint

Session Information

GET /api/session

Retrieves current session information.

Response (200 OK):

```
{  
  "session_id": "abc123def456",  
  "created_at": "2025-09-29T10:30:00",  
  "total_analyses": 5  
}
```

Analysis History

GET /api/history

Retrieves user's analysis history (last 20 analyses).

Response (200 OK):

```
{  
  "session_id": "abc123def456",  
  "total_analyses": 5,  
  "analyses": [  
    {  
      "id": "550e8400-e29b-41d4-a716-446655440000",  
      "timestamp": "2025-09-29T10:35:00",  
      "text_preview": "Beginning of analyzed text...",  
      "classification": "AI-generated",  
      "confidence": 0.8234,  
      "analysis_type": "single_text",  
      "source_type": "text"  
    }  
  ]  
}
```

```
]
}
```

Retrieve Specific Analysis

GET /api/analysis/{analysis_id}

Retrieves complete details for a specific analysis.

Response (200 OK):

```
{
  "success": true,
  "analysis": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "timestamp": "2025-09-29T10:35:00",
    "text_preview": "Full text content...",
    "result": { /* Full analysis result */ },
    "analysis_type": "single_text"
  }
}
```

Error Response (404 Not Found):

```
{
  "error": "Analysis not found"
}
```

Clear History

DELETE /api/clear-history

Deletes all analyses in the current session.

Response (200 OK):

```
{
  "success": true,
  "message": "History cleared successfully"
}
```

Error Handling

All error responses follow this format:

```
{
  "error": "Detailed error message"
}
```

Rate Limiting

Currently no rate limiting is implemented. Production deployment should include rate limiting to prevent abuse.

CORS Configuration

The API accepts requests from configured frontend origins with credentials support enabled for session persistence.

Appendix C - Test Documentation

Model Testing Suite (test_model.py)

Test Environment: Custom Python test suite with Electra transformer model. Total: 9 tests.

Core Functionality Tests

1. **Basic Prediction** - Validates model produces valid probability outputs. PASSED.
2. **Confidence Prediction** - Tests enhanced prediction with confidence level categorization. PASSED.
3. **Batch Prediction** - Tests multiple text processing in single call. PASSED.
4. **Consistency Check** - Ensures same input produces same output (reproducibility). PASSED.

Edge Case Tests

5. **Empty Text Handling** - Validates graceful handling of empty input. PASSED.
6. **Very Short Text** - Tests behavior with minimal input (single word). PASSED.
7. **Very Long Text** - Validates handling of text exceeding model token limits. PASSED.
8. **Special Characters** - Tests handling of emojis, symbols, and Unicode characters. PASSED.

Classification Tests

9. **Known AI Text** - Validates detection of typical AI-generated patterns. PASSED.
10. **Known Human Text** - Validates detection of casual human writing. PASSED.

Model Test Results: 9/9 passed (100%).

API Testing Suite (test_api_func.py)

Test Environment: Python requests library with Flask development server. Total: 16 tests.

Authentication Tests

1. **Health Endpoint Requires API Key** - Verifies endpoint protection without API key. PASSED.
2. **Health Endpoint with Valid API Key** - Tests successful authentication. PASSED.

Core Analysis Tests

3. **Text Analysis with Valid Input** - Tests standard text analysis workflow. PASSED.
4. **Text Analysis with Empty Text** - Validates rejection of empty input. PASSED.
5. **Text Analysis with Very Short Text** - Tests minimum word count enforcement. PASSED.
6. **Text Analysis with Force Single Analysis** - Tests analysis mode override. PASSED.
7. **Very Long Text Rejection** - Validates maximum text length enforcement. PASSED.

File Upload Tests

8. **Form Data Submission** - Tests file upload and text extraction. PASSED.
9. **Invalid File Format** - Validates rejection of unsupported file types. PASSED.
10. **File Size Limit** - Tests maximum file size enforcement. PASSED.

Session Management Tests

11. **Session Endpoint** - Tests session information retrieval. PASSED.

12. **History Endpoint** - Tests analysis history retrieval. PASSED.
13. **Specific Analysis Retrieval** - Tests fetching individual analysis by ID. PASSED.
14. **Analysis Retrieval with Invalid ID** - Validates error handling for nonexistent IDs. PASSED.
15. **Clear History** - Tests deletion of all session analyses. PASSED.
16. **Verify History Clearance** - Confirms history was successfully cleared. PASSED.

Error Handling Tests

17. **Invalid Endpoint Handling** - Tests 404 response for nonexistent endpoints. PASSED.
18. **Wrong HTTP Method** - Tests 405 response for incorrect HTTP methods. PASSED.

API Test Results: 16/16 passed (100%)

Frontend Component Testing (Storybook)

Test Environment: Storybook Test. Total: 36 tests.

Component Tests

1. **Button Component** - Primary, secondary, and disabled variants. 3/3 PASSED.
2. **FileAttachmentIndicator** - File attached and no file states. 2/2 PASSED.
3. **Header Component** - Default and responsive layouts. 2/2 PASSED.
4. **HighlightedText** - AI, human, mixed, and no highlighting. 4/4 PASSED.
5. **Icon Component** - Various icon types. 5/5 PASSED.
6. **Metric Component** - High, medium, and low confidence displays. 3/3 PASSED.
7. **Panel Component** - Navigation and active states. 2/2 PASSED.
8. **TextInput Component** - Empty, with text, with file, and disabled states. 4/4 PASSED.
9. **AITextDetectorPage** - Initial load, with results, and loading state. 3/3 PASSED.
10. **HistoryPage** - Empty, with analyses, detail view, and responsive. 4/4 PASSED.
11. **HomePage** - Default render and interaction. 2/2 PASSED.
12. **AnalysisHistory & HistoryTab** - List view and tab switching. 2/2 PASSED.

Frontend Test Results: 36/36 passed (100%)

Integration Testing

End-to-End Workflow Tests

1. **Complete Analysis Workflow** - User input through result display. PASSED.
2. **File Upload Workflow** - File selection through analysis completion. PASSED.
3. **Session Persistence** - Session maintained across page reloads. PASSED.
4. **History Tracking** - Multiple analyses tracked correctly. PASSED.

Integration Test Results: 4/4 passed (100%).

Overall Test Summary

Total Tests Executed: 61 tests

- Model Tests: 9/9 passed.
- API Tests: 16/16 passed

- Component Tests: 36/36 passed.
- Integration Tests: 4/4 passed

Success Rate: 100%

Performance Metrics:

- Single text analysis: 200-400ms.
- Sentence-level analysis (5 sentences): 800-1500ms
- File upload + analysis: 500-1200ms.
- Memory usage: Stable at ~400MB

References

Hugging Face Team. (2023) Transformers: State-of-the-art Machine Learning for PyTorch, TensorFlow, and JAX. <https://huggingface.co/transformers/>

Clark, K., Luong, M. T., Le, Q. V., & Manning, C. D. (2020) ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. International Conference on Learning Representations.

Redis Labs. (2023) Redis Documentation. <https://redis.io/documentation>

React Team. (2023) React Documentation. <https://react.dev/blog/2023/03/16/introducing-react-dev>

Storybook Team. (2024) Storybook Documentation <https://storybook.js.org/docs/get-started/frameworks/react-vite>

Flask Development Team. (2023) Flask Documentation. <https://flask.palletsprojects.com/>